

## Practical 9: -In-Memory Caching and Query Optimization

|                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objective:</b>         | To implement server-side caching and measure its impact on API response time.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Problem Statement:</b> | <ul style="list-style-type: none"><li>• Add in-memory caching to the Task Management backend using node- cache.</li><li>• Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds).</li><li>• Invalidate the cache on any write operation POST, PUT, or DELETE so stale data is never served.</li><li>• Record and compare API response times with and without caching using Postman or Thunder Client.</li><li>• Document the measured difference with at least 3 sample readings for each case (cached vs uncached).</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Implementation</b>     | <p>Cache Code:</p> <pre>import NodeCache from 'node-cache';  /**  * In-Memory Cache Service using node-cache  *  * Configuration:  * - stdTTL: 60 seconds (standard expiration window)  * - checkperiod: 120 seconds (background expired-key garbage collection)  */ const nodeCache = new NodeCache({ stdTTL: 60, checkperiod: 120 });  // Telemetry counters for cache performance tracking (Supplementary Problem 2) const counters = {   hits: 0,   misses: 0,   sets: 0,   invalidations: 0, };  export const cache = {   /**    * Retrieves an item from cache. Tracks hits/misses.    * @param {string} key    * @returns {any null}    */   get: (key) =&gt; {     const value = nodeCache.get(key);     if (value !== undefined) {       counters.hits += 1;       return value;     }     counters.misses += 1;     return null;   },    /**    * Stores an item into cache with standard or custom TTL.    * @param {string} key    * @param {any} val    * @param {number} [ttl]    * @returns {boolean}    */ }</pre> |

```
*/
set: (key, val, ttl) => {
  counters.sets += 1;
  if (ttl !== undefined) {
    return nodeCache.set(key, val, ttl);
  }
  return nodeCache.set(key, val);
},

/**
 * Invalidates a specific key from cache.
 * @param {string|string[]} key
 * @returns {number}
 */
del: (key) => {
  counters.invalidations += 1;
  return nodeCache.del(key);
},

/**
 * Invalidates all keys starting with a given prefix.
 * @param {string} prefix
 * @returns {number}
 */
delPrefix: (prefix) => {
  const keys = nodeCache.keys();
  const matched = keys.filter((k) => k.startsWith(prefix));
  if (matched.length > 0) {
    counters.invalidations += matched.length;
    return nodeCache.del(matched);
  }
  return 0;
},

/**
 * Clears the entire cache.
 */
flush: () => {
  counters.invalidations += nodeCache.keys().length;
  return nodeCache.flushAll();
},

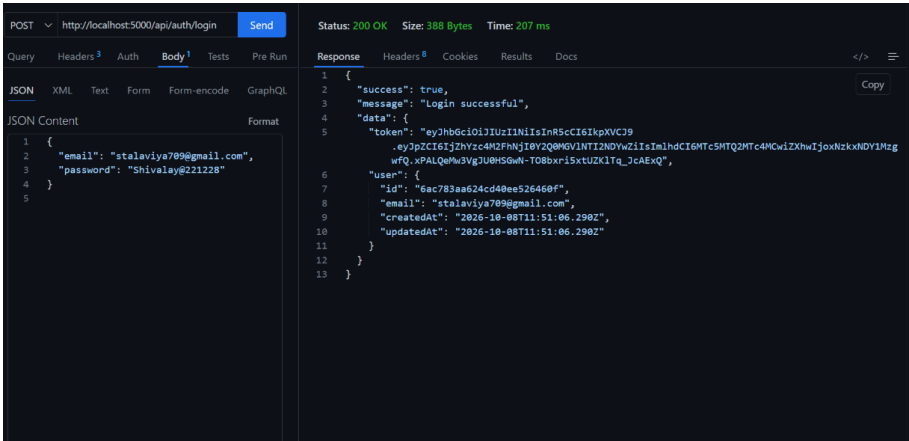
/**
 * Returns telemetry metrics (hit/miss counts, ratios, active keys).
 * Used by the debug endpoint.
 */
getMetrics: () => {
  const total = counters.hits + counters.misses;
  const hitRatio = total > 0 ? `${((counters.hits / total) *
100).toFixed(2)}%` : '0.00%';
  const keys = nodeCache.keys();

  return {
```

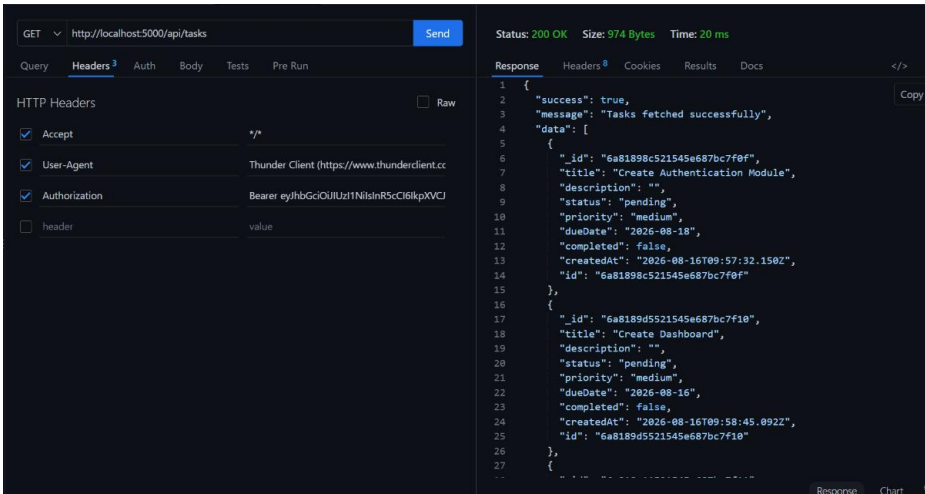
```
hits: counters.hits,
misses: counters.misses,
sets: counters.sets,
invalidations: counters.invalidations,
hitRatio,
totalRequests: total,
activeKeysCount: keys.length,
keys,
stdTTL: 60,
  };
},

/**
 * Direct access to underlying NodeCache instance.
 */
raw: nodeCache,
};

export default cache;
```



From the Cache:-



Outcome

Students will be able to implement in-memory caching for a REST API, ensure cache correctness through invalidation, and measure the performance impact using real response time data.

